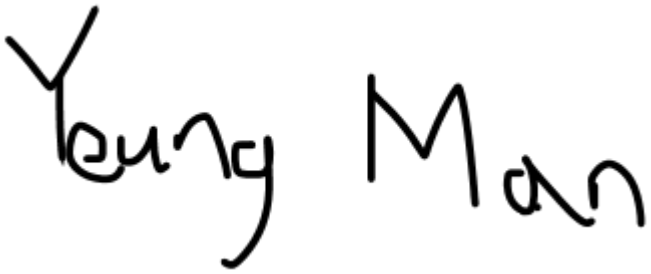
IEGR4320 Data Science in Practice (2025-26)
Assignment 3
Expected time: 6 hours



Learning outcomes:
1. To understand the basic machine learning algorithms, including classification, clustering, regression and reinforcement learning. 2. To practise a simple ANN model.

Instructions:

1. Do your own work. You are welcome to discuss the problems with your fellow classmates. Sharing ideas is great, and do write your own explanations.
2. If you use help from the AI tools, e.g. ChatGPT, DeepSeek and Gemini, write clearly how much you obtain help from the AI tools. No marks will be taken away for using any AI tools with a clear declaration.
3. All work should be submitted onto the blackboard before the due date.
4. You are advised to submit the following two items.
   a. A .pdf file containing the answers of the written part.
   b. A Assignment3_1155xxxxxx.py file storing all your programs for the five problems. (1155xxxxxx refers to your student ID)
5. Do type/write your work neatly. If we cannot read your work, we cannot grade your work.
6. We will grade your work based on Anaconda Python version 1.13.
7. The sample output in the specification is run based on Windows. Your output may be different if you run the programs on Mac or Linux machines.
8. If you do not put down your name, student ID in your submission (both .pdf and .py), you will receive a 10% mark penalty out of the assignment 3.
9. Due date:
   a. Session A: TBC January, 2026 (Thursday) 23:59
   b. Session B: TBC April, 2026 (Saturday) 23:59

Short questions (25%)
Answer all questions.

1. You are developing an AI-driven game agent for Tencent Games' chess platform, utilizing reinforcement learning and decision-making algorithms to enhance strategic gameplay and simulate human-like competitive behavior.

   a. What is reinforcement learning? (2%)

Reinforcement learning is a type of machine learning where an agent learns to make decisions by interacting with an environment, receiving rewards or penalties based on its actions.

   b. Why is reinforcement learning applied for training the game agents? (1%)

Reinforcement learning is applied to train game agents because it enables them to learn optimal strategies through trial and error, adapting to dynamic environments and mimicking human decision-making by maximizing long-term rewards.

   c. How can we categorize reinforcement learning? Explain briefly. (4%)

- **Value-Based**: Agent learns to estimate the value of actions (e.g., Q-learning), focusing on maximizing expected rewards.
- **Policy-Based**: Agent directly optimizes a policy that defines action probabilities (e.g., REINFORCE), suitable for continuous actions.
- **Actor-Critic**: Combines value-based and policy-based methods, using a value function (critic) to guide a policy (actor), improving stability and efficiency (e.g., A3C).

   d. Suppose you are training a game agent for a chess game, what are the states in the game? Explain your answer with one of the examples below. (3%)
   In this sub-problem, you can decide the chess game (example) as either (i) Chinese chess game, (ii) GO chess game, or (iii) international chess game.

In **international chess**, a state is the complete board configuration, including piece positions, current player, castling availability and move history for repetition rules.

Example: Initial position:

- Board: 8x8 with 32 pieces (e.g., White: RNBQKBNR on rank 1, pawns on rank 2; Black mirrored).
- White to move, all castling available, no en passant.
- Represented as a tensor (e.g., 8x8x12 for pieces, plus binary planes for metadata). This defines the game state for the RL agent to select moves.

   e. Based on the same chess game you chose in part (d), what are the actions in the game? Explain your answer with examples. (3%)

- **Piece Moves**: Move a piece to a legal square (e.g., from state s0 , initial position, White can play e2-e4: pawn moves to e4).
- **Captures**: Move a piece to capture an opponent's piece (e.g., if Black plays ...d5, White can play e4xd5, capturing the pawn).
- **Special Moves**:
- **Castling**: King moves two squares toward rook, rook jumps over (e.g., O-O, kingside castling: Ke1-g1, Rf1-f1).
- **En passant**: Pawn captures opponent's pawn that advanced two squares (e.g., if Black plays ...c5-c3, White pawn on d3 can capture d3xc4).
- **Promotion**: Pawn reaching 8th rank promotes to queen, rook, bishop, or knight (e.g., pawn a7-a8=Q).

f. Based on the same chess game you chose in part (d), how can you design the value-based reward? Explain your answer with examples. (3%)

- **Win**: Deliver checkmate (e.g., queen moves to h7, checkmate): reward = +1.
- **Loss**: Opponent checkmates you: reward = -1.
- **Draw**: Game ends in stalemate or threefold repetition: reward = 0.

g. Based on the same chess game you chose in part (d), how can you design the policy-based reward? Explain your answer with examples. (3%)

- **Win**: If the agent delivers checkmate (e.g., moving queen to h7 for checkmate), it receives a reward of +1 at the game's end.
- **Loss**: If the opponent checkmates the agent, it receives a reward of -1.
- **Draw**: If the game ends in a stalemate or threefold repetition, the reward is 0.
- **Non-terminal moves**: No immediate reward (reward = 0) during the game; the policy is updated based on the final outcome, backpropagating to earlier moves.

h. When training a game agent as an opponent, will the game agent always learn a better game strategy? Explain briefly. (3%)

When training a game agent as an opponent in international chess, it does not always learn a better strategy. The agent's improvement depends on factors like training setup and opponent quality. Self-play (e.g., AlphaZero) often leads to strong strategies by exploring diverse scenarios and refining policies. However, if trained against weak or repetitive opponents, the agent may overfit to their patterns, learning suboptimal strategies. For example, if the opponent always opens with 1. f3, the agent might excel against that but fail against stronger openings like 1. e4. Sufficient exploration, quality opponents, and robust algorithms (e.g., MCTS with policy gradients) are critical for consistent improvement.

i. When training a game agent as an opponent, do we set the goal as training a super skillful and perfect game agent? Explain briefly. (3%)

In international chess, the goal of training a game agent as an opponent is not always to create a super skillful and perfect agent. The objective depends on the purpose. For research or competitive platforms (e.g., Tencent’s chess), the aim might be a highly

skilled agent, like AlphaZero, that masters complex strategies through self-play and optimization. However, for user-facing applications, the goal could be an agent that provides a fun, adaptive challenge for players of varying skill levels, not necessarily perfection. For example, training an agent to mimic human-like play (with occasional mistakes) for casual players on a platform that prioritizes engagement over flawless performance. Thus, the goal varies—super skill for cutting-edge AI, balanced challenge for user enjoyment.

Practical problems (75%)
Work on the following five problems in a single .py file.
If your Python program cannot be run, you will receive no scores for the problem.
Put down your student ID as a comment in your first line of the program.

2. Write a function that trains a feed-forward neural network regressor and reports the mean square error based on the testing set. (15%)

   a. The architecture of the feed-forward neural network regressor is provided.
   b. Train a scaler based on training data, and apply the trained scaler on the testing data.
   c. Fill in the training loop for the feed-forward neural network regressor.
   d. Evaluate the mse based on the testing set

   This problem covers artificial neural networks and regression.

   Code:

```
# <Your student ID>
import numpy as np
import pandas as pd
from sklearn.datasets import fetch_california_housing
import torch
import torch.nn as nn
import torch.optim as optim
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
# Problem 2
# Define the feedforward neural network regressor
class FeedForwardNN(nn.Module):
    def __init__(self, input_size):
        super().__init__()
        self.fc1 = nn.Linear(input_size, 64)
        self.fc2 = nn.Linear(64, 32)
        self.fc3 = nn.Linear(32, 1)

    def forward(self, x):
        x = torch.relu(self.fc1(x))
        x = torch.relu(self.fc2(x))
        x = self.fc3(x)
        return x

def problem_2(X, y, test_size=0.3, learning_rate=0.01,
max_epochs=50000):
    testing_mse = 0
    # use this seed number for reproducibility
    # however, results from win, mac, linux are different
    # even they share the same seed number
    seed_number=5731
    torch.manual_seed(seed_number)
    np.random.seed(seed_number)
    X_train, X_test, y_train, y_test = train_test_split(X, y,
```

```
test_size=test_size, random_state=seed_number)
    scaler = StandardScaler()

    # write your logic here: train a scaler based on training data,
    # and apply the trained scaler on the testing data
    X_train = X_train
    X_test = X_test

    input_size = X_train.shape[1]
    model = FeedForwardNN(input_size)
    criterion = nn.MSELoss()
    optimizer = optim.Adam(model.parameters(), lr=learning_rate)

    X_train_tensor = torch.FloatTensor(X_train)
    y_train_tensor = torch.FloatTensor(y_train).view(-1, 1)
    X_test_tensor = torch.FloatTensor(X_test)
    y_test_tensor = torch.FloatTensor(y_test).view(-1, 1)

    for epoch in range(max_epochs):
        model.train()
        # write your logic here: fill in the training loop

    # write your logic here: evaluate the testing_mse based on
    # the testing set
    testing_mse = 0


    return testing_mse
```

Execution:

```
> data = fetch_california_housing()
> X, y = data.data, data.target
> mse = problem_2(X, y, test_size=0.3, learning_rate=0.01,
max_epochs=100)
> print("MSE in Testing: ", mse)
MSE in Testing:  0.3712635338306427
```

3. Write a function to perform k-fold cross validation using Naive Bayes classifier. Report cross validation accuracy and overall trained Naive Bayes classifier. (15%)

   a. Separate the dataset to k folds.
   b. Repeat the training for k fold times, using the (k-1) folds to train the Naive Bayes classifier, and accumulating the number of correct predictions of the left-out fold.
   c. Calculated the k-fold cross validation accuracy based on the accumulated number of correct predictions.
   d. Re-train the Naive Bayes classifier using all data, and report this re-trained model.
   e. If a random number is used, it is better to set the global random state to 5731 before running the random process (to promote reproducibility of the result).
   f. As there are many different approaches to divide the dataset into k folds, it is fine as long as (i) your cv accuracy is calculated based on the right procedure, and (ii) your reported cv accuracy does not differ a lot from our sample.

   This problem covers cross validation and classification.

   Code:

```
# Problem 3
from sklearn.datasets import load_wine
from sklearn.naive_bayes import GaussianNB
from sklearn import metrics
def problem_3(X, y, kfold=10):
    # write your logic here
    model = GaussianNB()
    cv_accuracy = 0
    seed_number = 5731  # default seed is 5731

    return cv_accuracy, model
```

   Execution:

```
> data = load_wine()
> X, y = data.data, data.target
> cv_accuracy, model = problem_3(X, y, kfold=5)
> print("5-fold cross validation accuracy:", cv_accuracy)
5-fold cross validation accuracy: 0.9719101123595506

> print("model description:", model)
model description: GaussianNB()
```

4. This problem defines the autoencoder for images with the following architecture. You only need to write the class for the autoencoder. (15%)
   - Input: d = input_size
   - Encoder: d → 512 → 256 → hidden_size
   - Latent Space: hidden_size
   - Decoder: hidden_size → 256 → 512 → d
   - Output: d

   This problem covers image data and autoencoders.

   Code:

```
# Problem 4
# write your logic here
# define the Autoencoder according the specification
class Autoencoder(nn.Module):
    def __init__(self, input_size, hidden_size):
        super().__init__()
        self.encoder = nn.Sequential(

        )
        self.decoder = nn.Sequential(

        )

    def forward(self, x):
        encoded = self.encoder(x)
        decoded = self.decoder(encoded)
        return decoded, encoded
```

   Execution:

```
> input_size = 28 * 28
> latent_size = 128
> model = Autoencoder(input_size, latent_size)
> print("Model information:", model) ", w)
Model information: Autoencoder(
  (encoder): Sequential(
    (0): Linear(in_features=784, out_features=512, bias=True)
    (1): ReLU()
    (2): Linear(in_features=512, out_features=256, bias=True)
    (3): ReLU()
    (4): Linear(in_features=256, out_features=128, bias=True)
  )
  (decoder): Sequential(
    (0): Linear(in_features=128, out_features=256, bias=True)
    (1): ReLU()
    (2): Linear(in_features=256, out_features=512, bias=True)
    (3): ReLU()
    (4): Linear(in_features=512, out_features=784, bias=True)
    (5): Sigmoid()
  )
```

```
)
```

5. This problem is the continuation of problem 4. The problem trains the autoencoder based on the FasionMINST images. Write a function that uses the trained autoencoder to convert all the images, batch-by-batch, to latent spaces. Return the latent space of all the images in a 2D NumPy array. (15%)

   This problem covers data loaders and image descriptors from latent space.

   Code:

```
# Problem 5
from torchvision import datasets, transforms
from torch.utils.data import DataLoader
def problem_5(model, loader):
    # write your logic here
    return_array = np.array([0])


    return return_array
```

   Execution:

```
> num_epochs = 10
> batch_size = 64
> learning_rate = 0.001
> criterion = nn.BCELoss()
> optimizer = optim.Adam(model.parameters(), lr=learning_rate)
> transform = transforms.Compose([
..     transforms.ToTensor(),
..     transforms.Lambda(lambda x: x.view(-1)),
..])
> images = datasets.FashionMNIST(root='data', train=True,
transform=transform, download=True)
> loader = DataLoader(dataset=images, batch_size=batch_size,
shuffle=False)
> for epoch in range(num_epochs):
>     for data in loader:
>         img, _ = data
>         output = model(img)[0]
>         loss = criterion(output, img)
>         optimizer.zero_grad()
>         loss.backward()
>         optimizer.step()
> image_descriptor = problem_5(model, loader)
> print("Image descriptor:", image_descriptor)
Image descriptor: [[ 0.18546753  0.31667182  0.7107127  ...
0.42309213  1.1096596
   1.0043461 ]
```

```
 [ 0.27696484 -0.03877646 -0.06742051 ...  1.6540381  -0.05599143
   1.8486406 ]
 [ 0.01850912 -0.22367409 -0.6537099  ... -0.09741548  0.14088869
   1.2497594 ]
 ...
 [ 0.12051444 -0.03620827 -1.1494102  ...  0.09235556 -0.80279666
   1.2451018 ]
 [ 0.27506077  0.32486653  0.08904518 ...  0.26045245 -0.22062412
   1.6328031 ]
 [-0.5056783   0.44087973  0.06432158 ... -0.48393807 -0.3527096
   0.85001457]]

> print("Dimension:", image_descriptor.shape)
Dimension: (60000, 128)
```

6. This problem is the continuation of problem 5. Write a function that performs k-means clustering on the image descriptors, and reports the centres of the clusters (i.e. visual bag of words of the list of the images). (15%)

   This problem covers visual bags of words and clustering.

   Code:

```
# Problem 6
from sklearn.cluster import KMeans
def problem_6(image_descriptor, k=5):
    # write your logic here
    centres = np.array([0])


    return centres
```

   Execution:

```
> centres = problem_6(image_descriptor, k=10)
> print("The 10 centres of the images:", centres)
The 10 centres of the images: [[ 0.1523784  -0.1430942
0.19136009 ... -0.24308091 -0.15820853
   0.75815135]
 [ 0.11580988 -0.20008475 -0.38598564 ...  0.41857797 -0.23624276
   1.2733756 ]
 [ 0.41715813  0.19927663  0.10397825 ... -0.22732991  0.38906255
   0.59773445]
 ...
 [-0.02764684 -0.2833568   0.06069239 ... -0.18634199 -0.34311098
   1.9914515 ]
 [ 0.08501321 -0.19006202 -0.07854423 ...  0.12056059 -0.29953244
   1.8155105 ]
 [ 0.6698747   0.55688316 -0.18796912 ... -1.4539495  -0.2974214
   0.97615856]]

> print("Dimension:", centres.shape)
Dimension: (10, 128)
```

More to know about question 4 - 6:

Using the classic approaches, visual bags of words are computed based on the image descriptors obtained from classic image processing methods, e.g. SIFT, HOG, … etc. In problems 4, 5 and 6, we construct the image descriptors using autoencoders (i.e. the latent space retrieved from the `encoded` in the autoencoder).

Furthermore, you can create the visual bag of words for the testing set in FashionMINST. For each image,

- Obtain latent space of the testing images based on the autoencoder
- Assign each image descriptor to its nearest centre obtained in problem 6
- Calculate the fraction of the assigned image descriptors to each centre

Hence, one image can be represented by 10 fractions (histogram of the 10 centres).

< End of Assignment >